System Analysis Document (SA) — CliniFlow AI

Team Name : Dependency Free

Participant Name : Tang Shao Xian

1. Introduction

CliniFlow AI is an AI-powered clinical workflow platform designed to streamline and automate the end-to-end patient journey for small and medium-sized (SME) clinics in Malaysia. This System Analysis Documentation outlines the technical scope, architecture decisions, and design rationale behind the CliniFlow AI development.

1.1. Purpose

This system analysis document highlights the technical scope and design-related decisions behind the CliniFlow AI development.

The key elements of the system covered are as follows:

1. **Architecture:** CliniFlow AI uses a polyglot service architecture with a Spring Boot (Java) backend for business logic and a Python (FastAPI + LangGraph) agent service for AI orchestration. Z.AI GLM serves as the central reasoning engine powering all intelligent workflows.
2. **Data Flows:** How patient symptom data flows from the web UI, through the Spring Boot API gateway, to the Python agent service for GLM processing, gets stored in relational and graph knowledge base storage, and returns structured clinical documentation to the doctor.
3. **Model Process:** The model shows the end-to-end workflow of the core features: pre-visit symptom intake, AI-powered SOAP note generation during consultation, and post-visit patient summary generation.
4. **Role of Reference:** This document serves as a reference for understanding the overall high-level architecture, data flows, and technical decisions of the CliniFlow AI project.

1.2. Background

SME clinics in Malaysia face significant operational challenges due to high patient volumes, limited resources, and reliance on manual processes. Doctors spend 30–50% of their time on documentation rather than patient care, and fragmented systems lead to inconsistent patient data.

1. **Previous Version:** There is no previous version. Most Malaysian SME clinics currently rely on a mix of paper records and basic digital tools, with no integrated AI-driven workflow system.

2. **Changes in Major Architectural Components:** This is a greenfield project. The architecture introduces a novel polyglot design with Spring Boot (Java) handling DDD-structured business logic, and Python (FastAPI + LangGraph) handling all AI agent orchestration, including Z.AI GLM integration.
3. **New Capabilities:** AI-powered symptom intake agent, automated SOAP note generation from text/voice inputs, an adaptive rule learning component (feedback-driven rule updates from doctor edits), multi-hop patient reasoning via a graph knowledge base concept, and patient-friendly post-visit summary generation.

1.3. Target Stakeholder

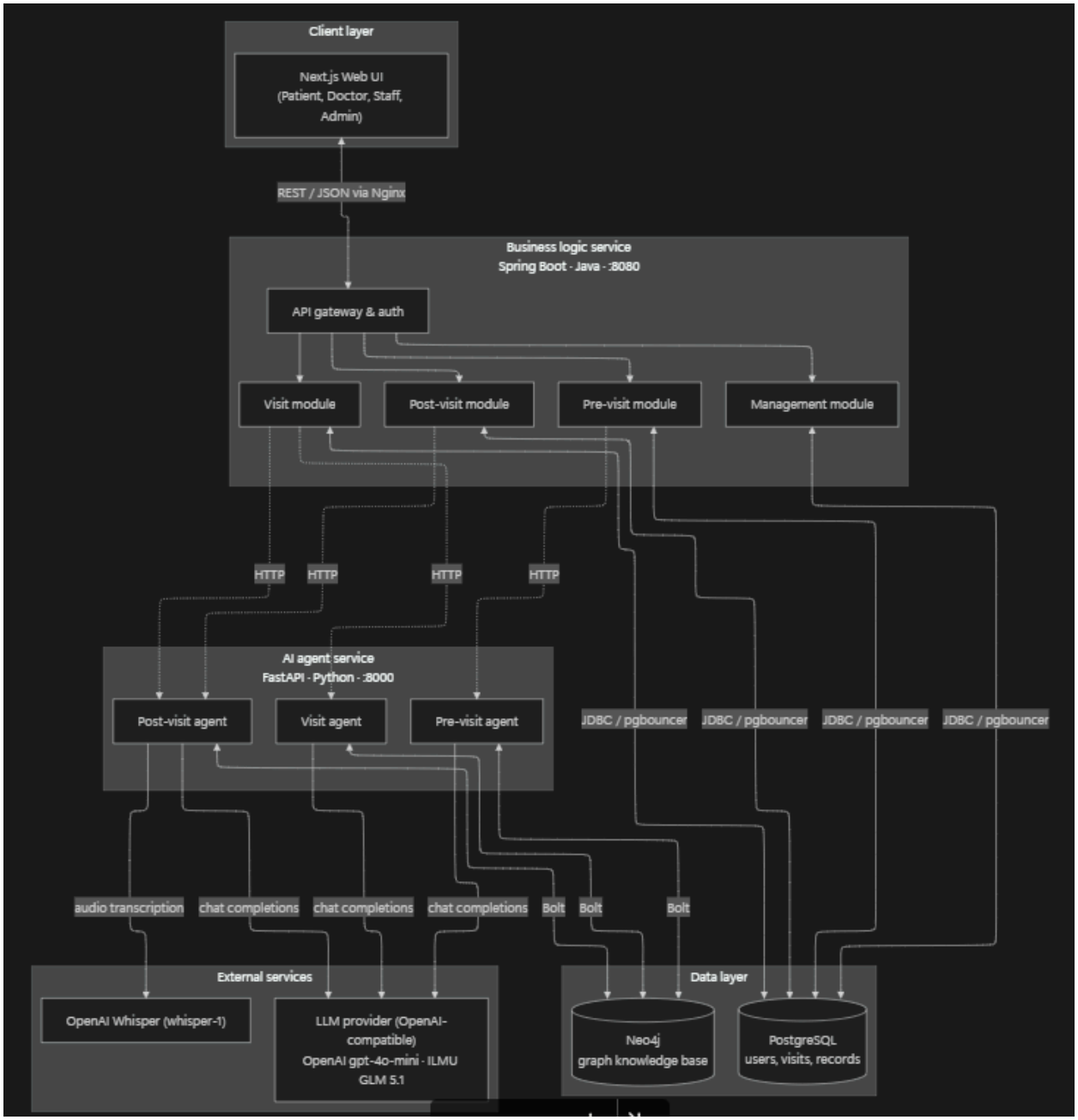
Stakeholders	Roles	Expectations
Clinic Doctors	Conduct consultations, review and edit AI-generated SOAP notes, validate clinical documentation	Reduced documentation time (≥50%), accurate AI-generated notes, easy review and editing workflow
Patients	Interact with symptom intake chatbot before visit, view post-visit records and medication instructions	Simple symptom reporting experience, clear and understandable post-visit instructions in plain language
Clinic Staff / Receptionists	Manage patient intake, access patient records, assist in administrative tasks	Pre-visit data available before appointment, structured and complete patient information
Clinic Administrators / Owners	Oversee operations, manage user accounts, view analytics	Centralized patient records, role-based access control, operational visibility

Development Team (Solo)	Build and maintain the full platform	Clear architecture documentation, modular codebase with DDD bounded contexts

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview



Type	Details
System	Web Application (Responsive)
Architecture	Polyglot Service Architecture (Spring Boot + Python Agent Service)

CliniFlow AI is structured as a polyglot client-server system with two backend services. The Spring Boot (Java) service handles API gateway responsibilities, authentication, role-based access control, patient record CRUD operations, and DDD-structured business logic across four bounded contexts (Pre-Visit, Visit, Post-Visit, Management). The Python (FastAPI + LangGraph) agent service handles all AI orchestration — agent workflows, LLM API calls, Speech-to-Text calls to OpenAI Whisper, graph knowledge base queries, and adaptive rule management. Both services share a Supabase-hosted PostgreSQL database for relational data and a Neo4j database for the graph knowledge base. The Next.js 14 (App Router) frontend communicates exclusively with the Spring Boot API. An Nginx reverse proxy fronts the stack on port 80, routing `/api/*` to Spring Boot and everything else to the Next.js server.

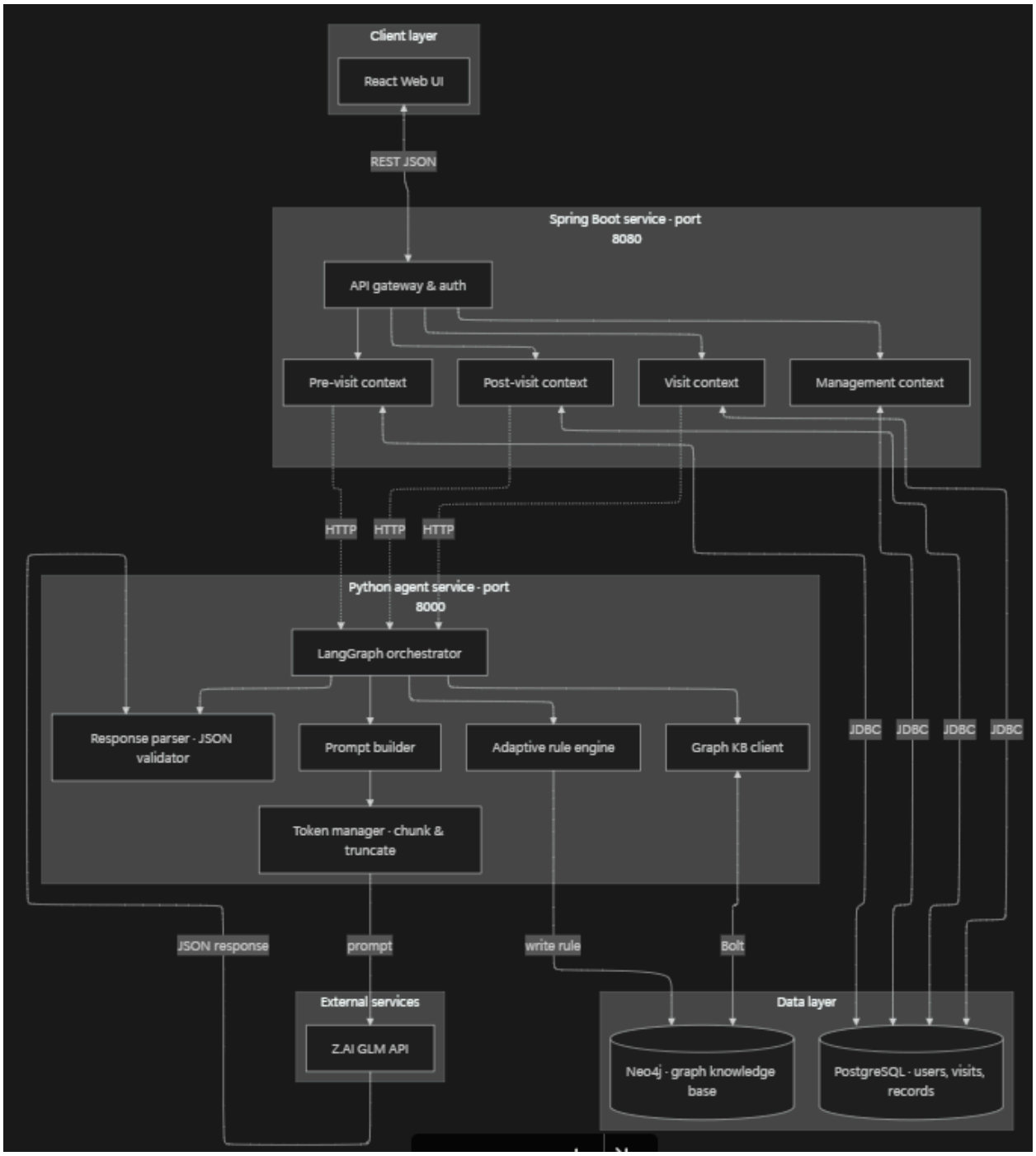
2.1.2. LLM as Service Layer

The LLM is integrated as the central reasoning engine within the Python agent service — not as a generic "AI Module." The provider is pluggable via an OpenAI-compatible HTTP client : OpenAI `gpt-4o-mini` for active dev, Z.AI GLM 5.1 via ILMU (`api.ilmu.ai/v1`, model `ilmu-glm-5.1`) for production. Speech-to-Text is a separate OpenAI Whisper integration (`OPENAI_BASE_URL`, `OPENAI_API_KEY`) so the LLM provider can swap without affecting transcription. The LLM powers three distinct agent workflows:

1. **Pre-Visit Agent:** Constructs prompts using patient history from the graph knowledge base + current symptom inputs. GLM generates follow-up questions and produces a structured pre-visit report.
2. **Visit Agent:** Constructs prompts using the pre-visit report + the transcribed consultation audio (from live recording or uploaded file) or manually typed text. GLM generates a structured medical report (following the SOAP format: Subjective, Objective, Assessment, Plan) with medical entity extraction. Adaptive (feedback-learned) rules are injected into the prompt context to adapt output formatting per doctor preferences.

3. **Post-Visit Agent:** Constructs prompts using the finalized medical report + medication data. GLM generates patient-friendly summaries with simplified medication instructions.

2.1.3. Dependency Diagram



The dependency and interaction flow between system components:

a. Prompt Construction: Each agent (Pre-Visit, Visit, Post-Visit) constructs prompts by combining: (1) system instructions defining the agent's role, (2) patient context retrieved from the graph knowledge base via multi-hop graph queries, (3) learned adaptive rules relevant to the current doctor/context, and (4) the user's current input (symptoms, consultation notes, etc.).

b. Context Window Contents: Patient demographics, visit history, known allergies, current medications, symptom descriptions, pre-visit report (for Visit agent), medical report (for Post-Visit agent), and applicable adaptive rules.

c. Response Parsing: GLM responses are parsed by LangGraph into structured JSON objects (e.g., medical report sections in SOAP format, medication lists, follow-up questions). These structured outputs are validated and passed back to Spring Boot via HTTP response.

d. Token Management: Input truncation is enforced at the Python agent layer. Patient history is summarized if it exceeds token limits. Conversation context is windowed for multi-turn symptom intake.

e. Major API Calls:

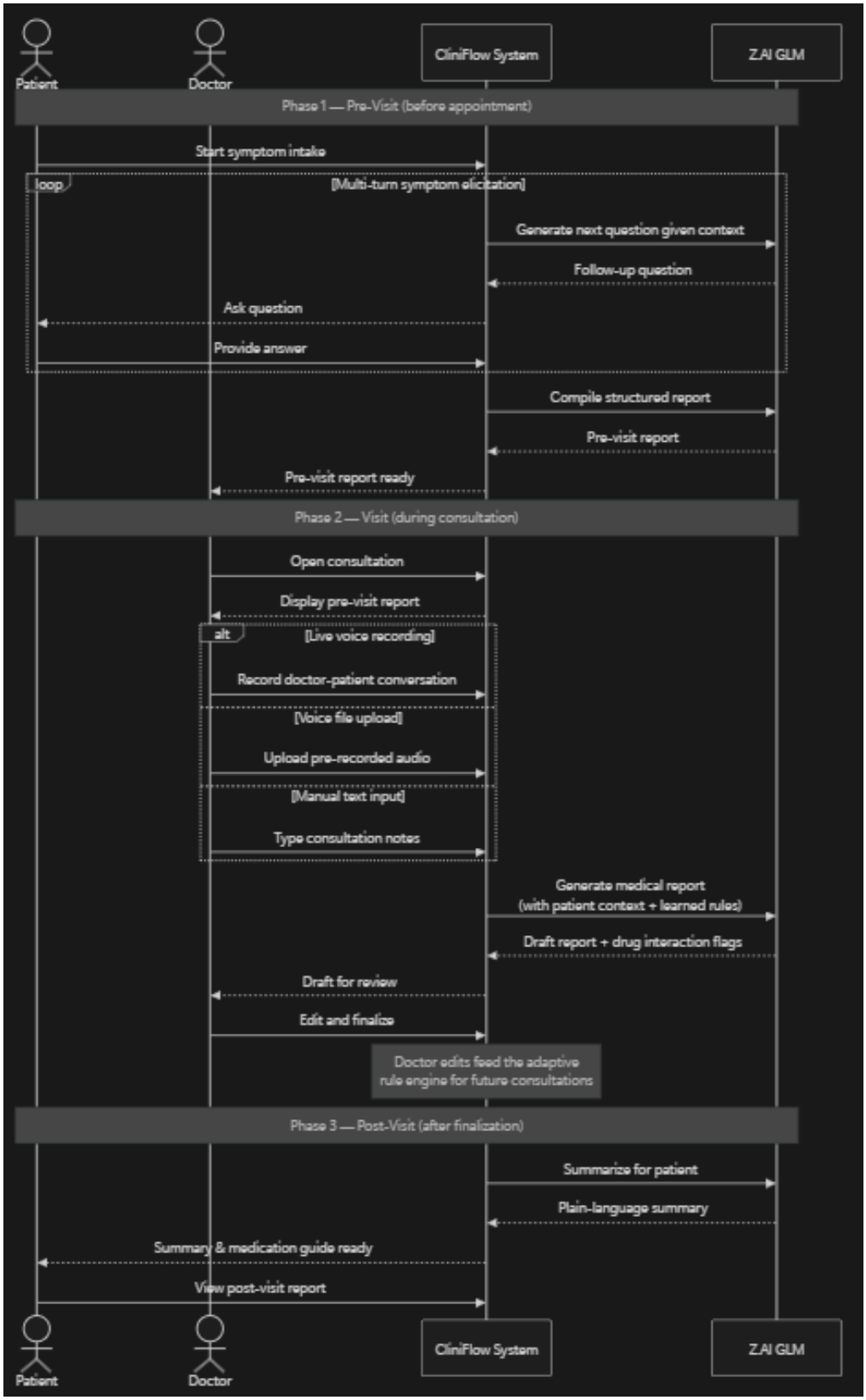
- React Web UI → Spring Boot API Gateway (REST/JSON, port 8080)
- Spring Boot Intake Service → Python Pre-Visit Agent (HTTP, port 8000)
- Spring Boot Consultation Service → Speech-to-Text service (HTTPS, external, for audio transcription)
- Spring Boot Consultation Service → Python Visit Agent (HTTP, port 8000)
- Spring Boot Summary Service → Python Post-Visit Agent (HTTP, port 8000)
- Python Agents → Z.AI GLM API (HTTPS, external)
- Python Agents → Neo4j (graph knowledge base, Bolt protocol)
- Spring Boot Services → PostgreSQL (JDBC)

f. Frontend-Backend-Service-Database Interaction:

- Frontend (React) communicates only with Spring Boot (REST API)
- Spring Boot handles auth, validation, and business logic, then delegates AI tasks to Python agent service via internal HTTP calls
- Python agent service queries the graph knowledge base (Neo4j) for patient context, calls GLM for reasoning, and returns structured outputs to Spring Boot
- Spring Boot persists results to PostgreSQL and returns responses to frontend

g. Distributed Components: The system runs as two independently deployable services (Spring Boot on port 8080, Python FastAPI on port 8000) sharing PostgreSQL and Neo4j databases. In production, these would be containerized (Docker) and deployed behind a reverse proxy.

2.2.2. Sequence Diagram



The high-level sequence above shows the complete business process across all three phases, including the multi-turn intake loop in Phase 1, the three input modes in Phase 2, and the adaptive rule feedback loop that lets the system learn from every consultation. This is the bird's-eye view at the business process level; detailed per-phase sequences with API endpoints, databases, and internal components are in Section 2.4 Design per Phase.

What this diagram communicates:

- *Intake is conversational, not a form.* The loop in Phase 1 shows GLM driving the dialogue — it decides when enough information has been gathered.
- *The doctor has choice in how to capture the consultation.* The `alt / else` block in Phase 2 makes the three input modes explicit at the business level.
- *The system learns from every visit.* The Note after Phase 2 highlights that doctor edits are not discarded — they become training signal that improves future reports.
- *GLM is load-bearing across all three phases.* Every phase contains at least one GLM call. Remove GLM and the system degenerates to a CRUD app.

2.2. Technological Stack

1. **Frontend:** Next.js 14 (App Router, React 18, TypeScript) — responsive web application for all user roles (doctor workstation, patient portal, admin dashboard)
2. **Backend (Business Logic):** Spring Boot 3.3 (Java 21) — RESTful API, DDD bounded contexts, auth/RBAC, patient record CRUD
3. **Backend (AI Orchestration):** FastAPI (Python 3.12) + LangGraph — agent workflows, LLM integration, Whisper STT client, graph KB client, adaptive rule engine
4. **Relational Database:** Supabase-hosted PostgreSQL (via pgbouncer transaction-mode pooler) — users, patient demographics, visit records, agent turns, audit log
5. **Graph Database:** Neo4j 5.20 Community — graph knowledge base (patient-symptom-diagnosis-medication relationships, adaptive rules). Schema applied at FastAPI startup from `agent/app/graph/schema.py`.
6. **LLM Provider:** Pluggable OpenAI-compatible endpoint — OpenAI `gpt-4o-mini` (dev), Z.AI GLM 5.1 via ILMU (prod)
7. **Speech-to-Text:** OpenAI Whisper (`whisper-1`), called by the Python agent
8. **Deployment:** Docker Compose — four containers (frontend, backend, agent, nginx; neo4j optional via `local-only` profile). Nginx reverse proxy on port 80 with `proxy_read_timeout 300s` for slow LLM paths.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

External Entities: Patient, Doctor, Clinic Staff, Clinic Admin

Processes:

- P1: Symptom Intake Process — receives patient symptoms, queries graph KB for history, calls GLM, generates structured pre-visit report
- P2: Consultation Documentation Process — receives doctor's consultation input (live voice recording, uploaded audio, or text), transcribes voice via STT, queries graph KB, applies adaptive rules, calls GLM, generates structured medical report
- P3: Post-Visit Summary Process — receives finalized medical report, calls GLM, generates patient-friendly summary
- P4: Adaptive Rule Learning Process — receives doctor edit diffs, analyzes patterns, proposes and stores rules in graph KB
- P5: User & Access Management — handles authentication, role-based access control

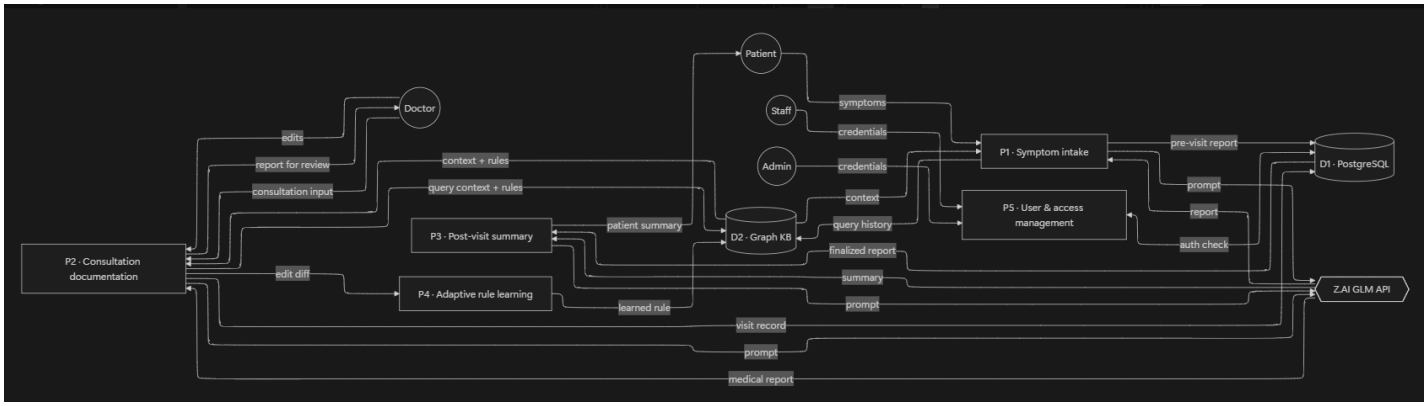
Data Stores:

- D1: PostgreSQL — patient records, visit records, user accounts
- D2: Neo4j (graph knowledge base) — patient knowledge graph, adaptive rules

Key Flows:

- Patient → P1: symptom text input → P1 → D2: query patient history → P1 → GLM: generate follow-up questions and report → P1 → D1: store pre-visit report
- Doctor → P2: consultation input (voice or text) → P2 → STT: transcribe voice → P2 → D2: query context + rules → P2 → GLM: generate medical report → P2 → Doctor: review → P2 → D1: persist visit
- P2 → P4: edit diff → P4 → D2: store learned rule
- P3 → Patient: simplified summary with medication instructions

No black holes exist — all processes have both inputs and outputs. No miracles exist — all outputs trace back to data inputs.



2.3.2. Normalized Database Schema

PostgreSQL (Relational — Supabase-hosted, 3NF):

Migrations live in `backend/src/main/resources/db/migration/` as reference-only SQL. Flyway is NOT used (incompatible with Supabase pgbouncer) — schema is applied manually via the Supabase SQL editor. JPA runs with `ddl-auto: none`.

Table	Key Columns	Relationships / Notes
users	id (PK), email (citext, UK), password_hash, role (PATIENT/DOCTOR/STAFF/ADMIN), full_name, is_active, consent_given_at, gmt_create, gmt_modified	1:N with visits (as doctor); V4 adds consent_given_at for PDPA
patients	id (PK), user_id (FK users), national_id_ciphertext (bytea, app-encrypted), national_id_fingerprint (HMAC-SHA256, UK), full_name, date_of_birth, gender, phone, email, gmt_create, gmt_modified	1:N with visits
visits	id (PK), patient_id (FK), doctor_id (FK), status (SCHEDULED/IN_PROGRESS/FINALIZED/CANCELLED), started_at, finalized_at, report_draft (jsonb), report_confidence_flags (jsonb), gmt_create, gmt_modified	Aggregate root. V6 adds report_draft/confidence_flags for Report Agent streaming state.
pre_visit_reports	id (PK), visit_id (FK, UK), structured (jsonb), source (PATIENT/AI/STAFF), gmt_create, gmt_modified	1:1 with visits

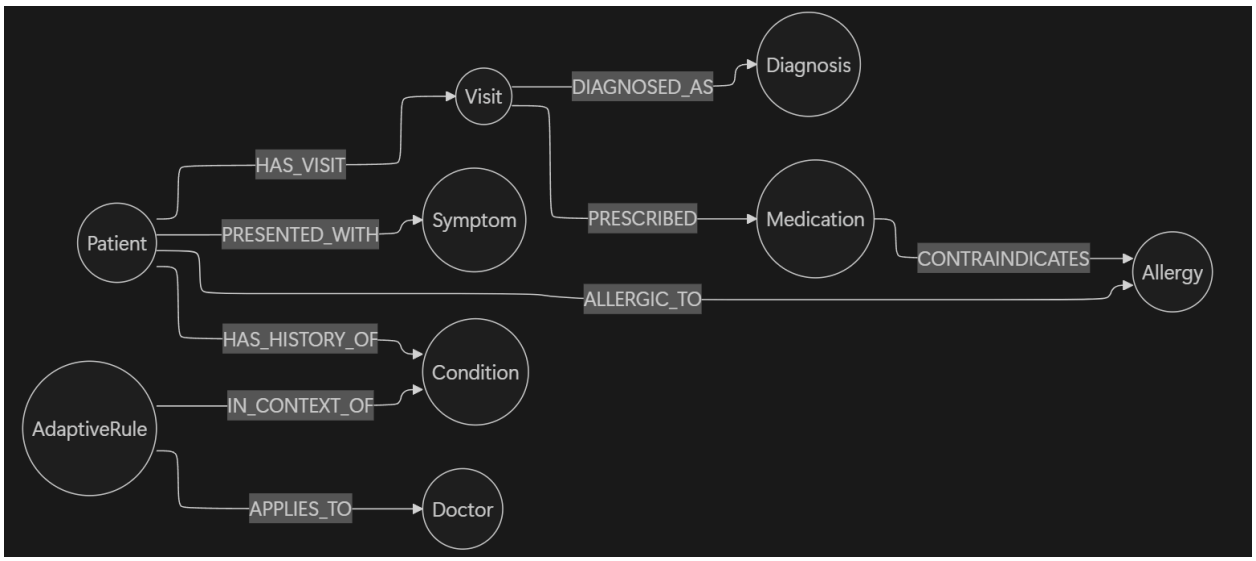
medical_reports	id (PK), visit_id (FK, UK), subjective, objective, assessment, plan, ai_draft_hash (sha256), is_finalized, finalized_by (FK users), finalized_at, preview_approved_at, summary_en, summary_ms, gmt_create, gmt_modified	1:1 with visits. V8 adds preview_approved_at + bilingual summary columns — patient-facing summaries are written HERE on finalize, not to post_visit_summaries. CHECK constraint: is_finalized ⇒ finalized_by + finalized_at set.
post_visit_summaries	id (PK), visit_id (FK, UK), patient_summary, summary_en, summary_ms, gmt_create, gmt_modified	LEGACY. New finalize writes to medical_reports.summary_en/summary_ms; this table is only read as a fallback for pre-refactor visits.
medications	id (PK), visit_id (FK), name, dosage, frequency, duration_days (int), instructions (text), gmt_create, gmt_modified	N:1 with visits. Written by ReportReviewAppService.finalize() from finalizedReport.plan.medications.
agent_turns	id (bigserial, PK), visit_id (FK), agent_type (pre_visit/report), turn_index (int), role (system/user/assistant/tool), content, reasoning, tool_call_name, tool_call_args (jsonb), tool_result (jsonb), created_at. UNIQUE(visit_id, agent_type, turn_index)	V5. Append-only agent conversation log — DB triggers block UPDATE/DELETE. V7 adds AFTER INSERT trigger that writes an audit_log row for each turn (PDPA).
audit_log	id (bigserial, PK), occurred_at, actor_user_id (FK users), actor_role, action (READ/CREATE/UPDATE/DELETE/LOGIN/EXPORT), resource_type, resource_id, correlation_id, payload_hash, ip_address (inet), metadata (jsonb)	PDPA append-only. DB triggers raise exception on any UPDATE or DELETE — never mutate from application code, always INSERT.

ERD — PostgreSQL schema (3NF):

Graph Knowledge Base (Neo4j):

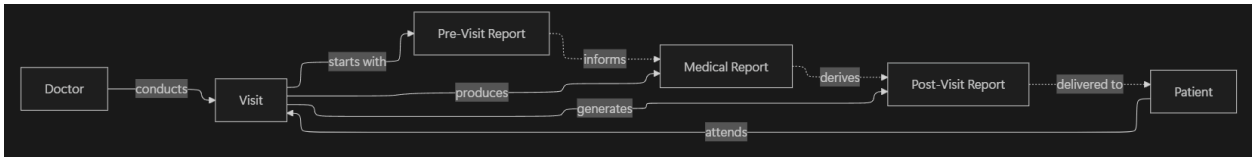
Node types: Patient, Visit, Symptom, Diagnosis, Medication, Allergy, Condition, AdaptiveRule

Key relationships: `(Patient)-[:PRESENTED_WITH]->(Symptom)`, `(Visit)-[:DIAGNOSED_AS]->(Diagnosis)`, `(Visit)-[:PRESCRIBED]->(Medication)`, `(Medication)-[:CONTRAINDICATES]->(Allergy)`, `(Patient)-[:HAS_HISTORY_OF]->(Condition)`, `(AdaptiveRule)-[:APPLIES_TO]->(Doctor)`, `(AdaptiveRule)-[:IN_CONTEXT_OF]->(Condition)`



2.3.3. Domain Relationship Map (Business View)

The graph schema above shows the *storage* view — every clinical concept is a node for query flexibility. At the *domain* level, the model is simpler: Visit is the aggregate root that owns its three phase artifacts, and clinical entities live either within the visit artifact that produces them or at the Patient level when they persist across visits.



2.4. Detail Design

The end-to-end clinical workflow is split into three phases, each owned by a bounded context and powered by a dedicated AI agent. Sections 2.4.1–2.4.3 describe each phase's data flow end-to-end; Sections 2.4.4–2.4.5 zoom into the internal architecture of the two Python agents that power those phases (Pre-Visit Intake Agent and Report Agent — the Post-Visit summary is produced by a tool call from the Report Agent, not a separate agent process).

2.4.1. Pre-Visit Phase Design

Context: Pre-Visit bounded context (Spring Boot) + Pre-Visit Agent (Python)

Actor: Patient

Trigger: Patient logs in before their appointment and starts symptom intake

Agent Prompt Composition:

- *System instructions:* agent role ("medical symptom intake assistant"), output schema, tone guidelines
- *Patient context from graph KB:* known allergies, chronic conditions, most recent visit summary (if any)
- *Conversation state:* symptoms reported so far in this session
- *Current input:* patient's latest message

Data Read:

- Graph KB — patient node, allergy and condition edges, prior visit summaries
- PostgreSQL — patient demographics

Data Written:

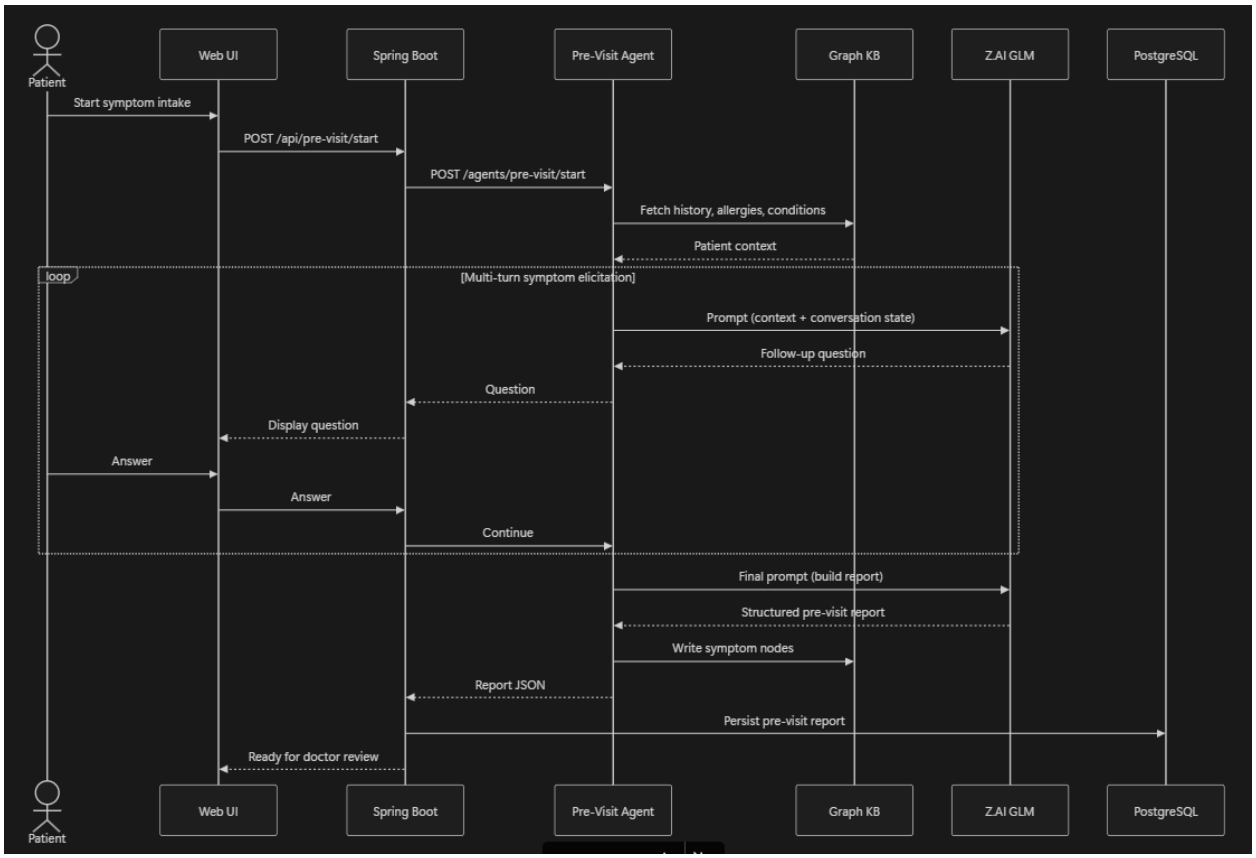
- PostgreSQL — pre-visit report (symptom summary, structured data JSON, generated timestamp)
- Graph KB — new symptom nodes attached to the patient

GLM Output Structure:

- Either: next follow-up question (string) with reasoning (why this question)
- Or: terminal signal with final structured pre-visit report (symptoms, duration, severity, relevant history flags)

Key Design Decision: The agent controls when intake is complete, not a fixed script. GLM decides "do I have enough information?" on each turn. This is multi-step reasoning, not template filling.

Sequence Diagram — Pre-Visit Flow:



2.4.2. Visiting Phase Design

Context: Visit bounded context (Spring Boot) + Visit Agent (Python)

Actor: Doctor (with the patient present)

Trigger: Doctor opens the consultation for a patient whose pre-visit report is ready

Input Modes: The doctor has three ways to capture the consultation, exposed as three tabs on the Consultation panel:

1. *Text tab (default)* — doctor types or pastes consultation notes directly. Fastest path and the fallback when audio tooling is unavailable.
2. *Voice tab — file upload* — doctor drags-and-drops (or picks) a pre-recorded audio file (MP3/WAV/M4A/WebM/FLAC/OGG, up to 25MB). Used when the doctor recorded on a separate device such as a phone at the bedside.
3. *Live tab — in-browser recording* — doctor clicks the record button to capture the consultation live via the browser's MediaRecorder API. A red pulsing timer shows elapsed time. On stop, the WebM blob is posted to the backend.

Both voice inputs (file upload and live recording) post to `POST /api/visits/{id}/audio` on Spring Boot, which forwards the multipart payload to the Python agent's `/agents/stt/transcribe` endpoint, which in turn calls OpenAI Whisper (`whisper-1`). The transcript is returned synchronously and auto-populates the Text tab, where the doctor can review and edit before clicking Generate report. Spring multipart limits are set to 25MB to match the Nginx `client_max_body_size`.

Agent Prompt Composition:

- *System instructions:* agent role ("clinical documentation assistant"), output schema using the SOAP format (Subjective, Objective, Assessment, Plan), medical entity extraction rules
- *Patient context from graph KB:* allergies, current medications, chronic conditions, recent diagnoses (multi-hop query)
- *Pre-visit report:* structured symptom summary from Phase 1
- *Adaptive rules:* formatting and content rules learned from this doctor's prior edits
- *Consultation input:* either the transcribed audio or the manually typed text

Data Read:

- PostgreSQL — visit record, pre-visit report, doctor profile
- Graph KB — patient context (multi-hop), applicable adaptive rules
- Speech-to-Text service — transcript produced from recorded or uploaded audio

Data Written:

- PostgreSQL — medical report with four SOAP-format sections, medication entries, finalized flag, reference to the source audio file (if voice)
- Graph KB — new diagnosis, prescription, and symptom nodes linked to the visit; doctor edit diffs for adaptive rule analysis

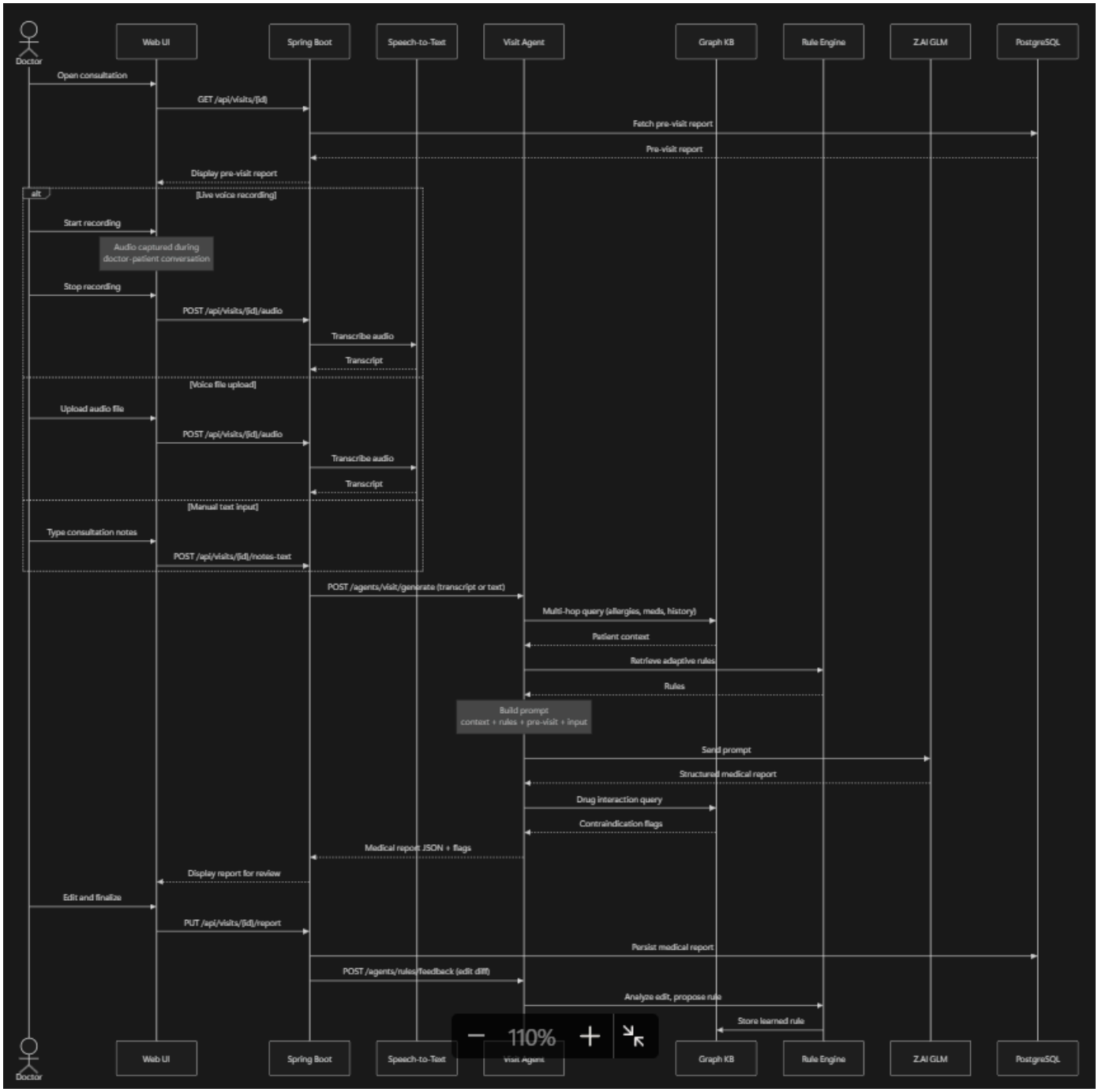
GLM Output Structure:

- Structured medical report with four sections: Subjective, Objective, Assessment, Plan
- List of extracted medications with dosage fields
- List of extracted diagnoses
- Missing-information flags (if any required fields could not be inferred from the consultation)

Key Design Decision: The doctor must review and finalize the report before it is persisted as the official record. The AI output is a draft, not a clinical decision. Every edit feeds the adaptive rule engine, which proposes new rules (subject to a confidence threshold) to apply in subsequent consultations.

Drug Interaction Check Sub-Flow: When a new prescription is extracted, the Visit Agent runs a separate graph query: `Prescription → contraindicated_by → Allergy` and `Prescription → interacts_with → ActiveMedication`. Any conflicts are surfaced to the doctor before the report is finalized.

Sequence Diagram — Visit Consultation Flow:



2.4.3. Post-Visit Phase Design

Context: Post-Visit bounded context (Spring Boot) + Post-Visit Agent (Python)

Actor: Patient (reads), system (generates)

Trigger: Doctor finalizes the medical report, emitting a `VisitFinalized` event

Agent Prompt Composition:

- *System instructions:* agent role ("patient-friendly summary writer"), output schema, reading-level target (layperson)
- *Finalized medical report:* full four SOAP-format sections
- *Medication list:* structured prescriptions
- *Patient context (optional):* age, language preference

Data Read:

- PostgreSQL — finalized medical report, medication list, patient demographics

Data Written:

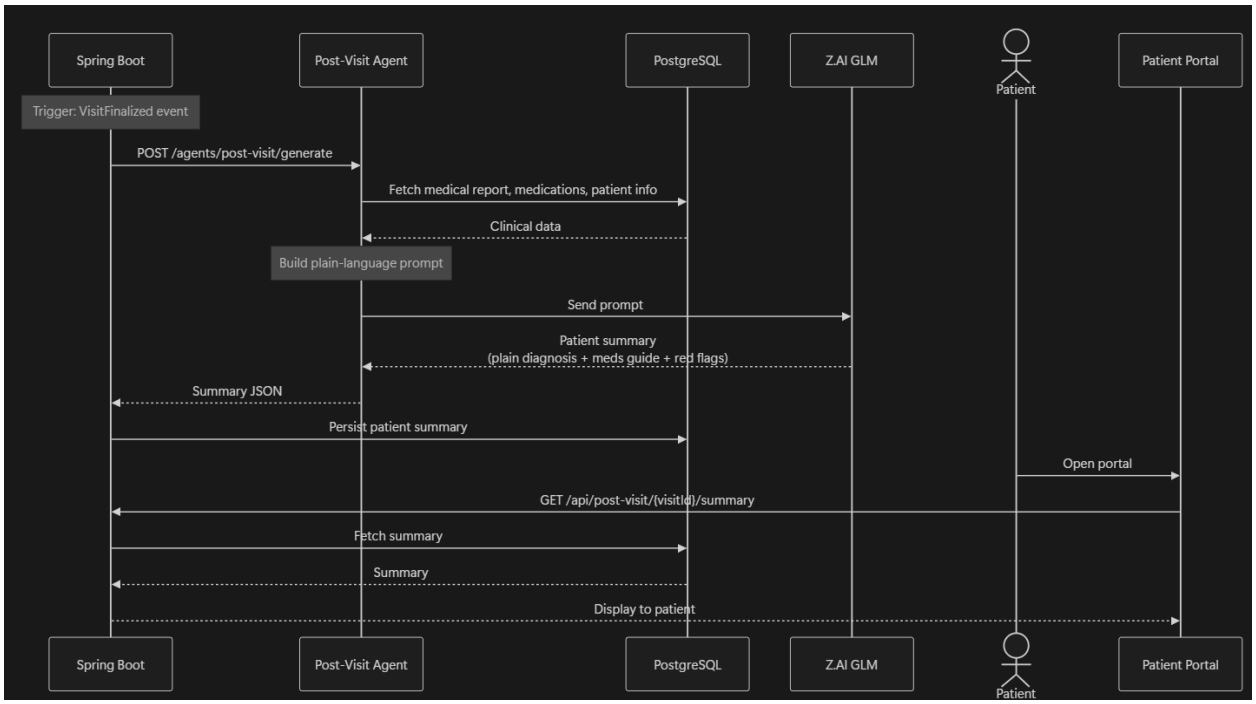
- PostgreSQL — patient summary (plain-language diagnosis, care instructions, red flags), structured medication guide

GLM Output Structure:

- Plain-language diagnosis explanation
- Structured medication guide (what, when, how long, why, with-food/empty-stomach notes)
- Red-flag symptoms (when to seek urgent care)
- Follow-up guidance

Key Design Decision: Output is generated once and cached — no need to re-run on every patient portal view. The medical report is the source of truth; the summary is a derived view. If the medical report is ever amended, the summary is regenerated.

Sequence Diagram — Post-Visit Flow:



2.4.4. Pre-Visit Agent Design

Purpose: Conversational symptom-intake agent. Collects chief complaint, duration, associated symptoms, allergies, and current meds — *confirming* facts already known from the graph KB rather than re-asking.

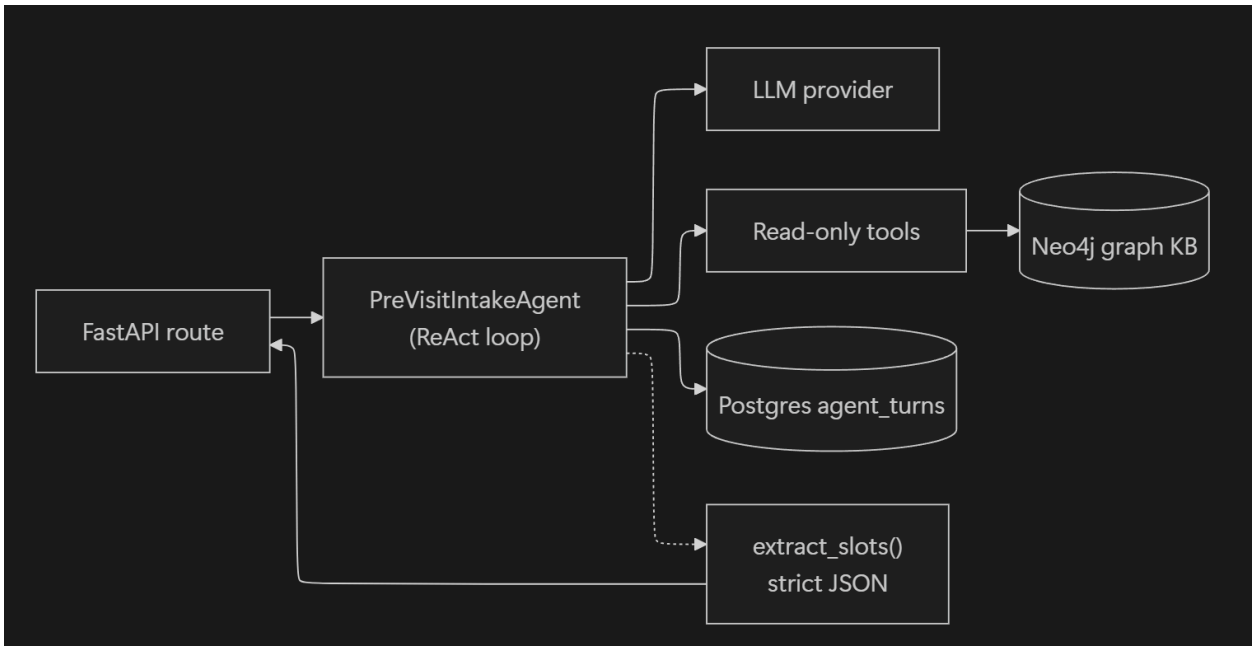
Class: `PreVisitIntakeAgent(BaseAgent)` · `agent_type = "pre_visit"`

Tools (all read-only):

Tool	Purpose
get_patient_context	Fetch demographics, allergies, active meds, chronic conditions from Neo4j
get_visit_history	Fetch recent finalized visits for the patient
emit_reasoning	Stream a short reasoning delta to the UI (non-visible turn)

Output paths: (1) a conversational stream of follow-up questions rendered in the patient portal chat, and (2) a post-hoc `extract_slots()` LLM call that returns a strict-JSON `PreVisitSlots` model. A parse failure in extraction returns an empty slots object rather than breaking the chat.

Pre-Visit Agent — High-Level Architecture:



Key design decisions:

- The agent, not a fixed script, decides when intake is complete.
- Slot extraction is decoupled from the chat loop — a bad JSON parse cannot corrupt patient-facing output.
- Read-only: persistence is the orchestrator's responsibility.

2.4.5. Report Agent Design

Purpose: Transforms a consultation transcript (Text / Voice upload / Live recording) into a structured SOAP report, and on finalize produces the bilingual patient summary. Unlike the Pre-Visit Agent, this agent **writes** — it persists the draft via `update_soap_draft`.

Class: `ReportAgent(BaseAgent)` · `agent_type = "report"`

Three modes (same ReAct loop, different input): *Generate* from transcript → *Edit* from doctor chat instruction (current draft is injected into the prompt) → *Finalize* produces bilingual EN/MS summary. Spring Boot owns the visit status transition; the agent only produces drafts + summaries.

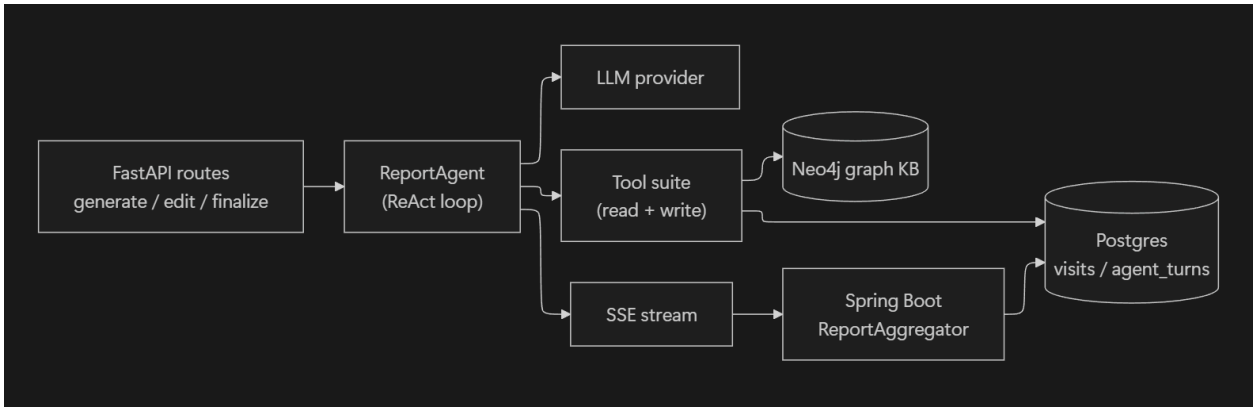
Tool allowlist:

Tool	Purpose	Reads / Writes

get_visit_history	Recent visits for the patient	Read: Postgres + Neo4j
clinical_dictionary_extract	Extract ICD-10 codes and clinical entities from transcript	Read-only LLM sub-call
update_soap_draft	Persist structured MedicalReport to `visits.report_draft` jsonb + `visits.report_confidence_flags`	Write: Postgres
record_inferred_edge	Record an inferred graph relationship with 0.0–1.0 confidence (Graphify pattern)	Write: Neo4j
emit_reasoning	Stream reasoning delta to UI	Side-channel SSE only

Two safeguards in the prompt: the `HERMES_FENCE` keeps adaptive rules in documentation-style territory only (never diagnosis, dosing, or red-flag thresholds), and the `EDIT PROCESS` block forbids `ask_doctor_clarification` on a doctor edit request.

Report Agent — High-Level Architecture:



Key design decisions:

- Doctor-in-the-loop is non-negotiable — the agent produces a draft; only an explicit finalize by a real doctor promotes it to the patient-facing record.
- State lives in Postgres, not agent memory — the draft is persisted on every `update_soap_draft` call, so a dropped connection never loses the working draft.
- Post-Visit is not a separate agent — the bilingual summary is a tool call the Report Agent makes at finalize time, keeping it in the same transaction as the report write.

3. Functional Requirements & Scope

This section highlights the core boundaries of the project, enabling focus on a high-impact MVP to showcase the technical feasibility of CliniFlow AI.

3.1. Minimum Viable Product

The MVP features are organised by phase to mirror the clinical workflow: pre-visit, visiting, and post-visit. This structure maps directly to the three bounded contexts in the system and the three AI agents that power them.

3.1.1. Pre-Visit Phase Features

#	Feature	Description
1.1	AI Symptom Intake Agent	Patients describe symptoms via a conversational chatbot. GLM dynamically generates follow-up questions based on prior responses, driving multi-turn symptom elicitation until sufficient information is captured.
1.2	Patient History Retrieval	Before generating follow-up questions, the Pre-Visit Agent queries the graph knowledge base for the patient's known allergies, chronic conditions, and prior visit summaries. This context shapes the questions asked and prevents repeat data entry.
1.3	Structured Pre-Visit Report Generation	The agent transforms the conversational intake into a structured report with symptom summary, severity indicators, duration, and relevant history flags. The doctor reviews this before the consultation.

3.1.2. Visiting Phase Features

#	Feature	Description
2.1	Live Voice Recording of Consultation	Doctor clicks Record to capture the live doctor-patient conversation on the spot. When the consultation ends, the audio is sent to a Speech-to-Text service and the transcript becomes the primary input to the Visit Agent. This is the primary path — doctors no longer type during consultation.

2.2	Alternative Input Modes (Voice Upload & Text)	For situations where live recording isn't practical, doctors can upload a pre-recorded audio file or type consultation notes manually. All three input modes converge on the same agent pipeline.
2.3	AI Medical Report Generation	The Visit Agent calls GLM to generate a structured medical report from the consultation input. The report follows the SOAP format (Subjective, Objective, Assessment, Plan) and includes extracted clinical entities.
2.4	Medical Entity Extraction	The agent identifies and structures key clinical entities from the consultation — symptoms with duration, diagnoses, prescribed medications with dosage, and proposed follow-up actions.
2.5	Missing Information Detection	The agent flags incomplete clinical information (e.g., prescribed medication without duration, assessment without supporting objective findings) and prompts the doctor to complete them before finalization.
2.6	Drug Interaction Check (Graph KB Multi-Hop Query)	When a new prescription is added, the system traverses the patient's knowledge graph to detect contraindications with existing medications or known allergies. This is the hero demo for the graph reasoning capability.
2.7	Doctor Review & Edit Workflow	The doctor reviews, edits, and finalizes the AI-generated medical report. The UI highlights AI-generated content versus doctor edits. Once finalized, the report is locked and persisted.
2.8	Adaptive Rule Learning	Every doctor edit is captured as a feedback signal. The rule engine analyzes edit patterns and proposes formatting/content rules that are injected into future prompts to adapt to the doctor's documentation style.

3.1.3. Post-Visit Phase Features

#	Feature	Description
3.1	Patient-Friendly Summary Generation	The Post-Visit Agent transforms the finalized medical report into plain-language patient instructions. Diagnosis is explained without jargon, and care instructions are simplified.
3.2	Structured Medication Guide	The agent produces a medication list with clear dosage, frequency, duration, and special instructions (e.g., "take with food") in language the patient can follow.

3.3	Red Flag & Follow-Up Guidance	The summary includes red-flag symptoms (when to seek urgent care) and follow-up appointment information, which the patient accesses via the patient portal.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	CliniFlow AI must handle concurrent consultations during peak clinic hours (morning rush).	Spring Boot thread pool configuration. Python async agent service with FastAPI. Connection pooling for PostgreSQL and Neo4j.
Reliability	AI-generated medical reports must always go through doctor review before finalization. Failed GLM API calls must not corrupt patient records.	Mandatory doctor confirmation step (US-D04). Database transactions with rollback on API failure. Graceful degradation: if GLM is unavailable, doctors can still enter notes manually.
Maintainability	Each bounded context (Pre-Visit, Visit, Post-Visit, Management) is independently modifiable. Agent logic is separated from business logic.	DDD package structure in Spring Boot. Python agent service with per-agent modules. Clear HTTP contract between Java and Python services.
Token Latency	GLM API calls must return a response within 5s under normal load (p95). Pre-visit intake follow-up questions must feel conversational (<3s).	Timeout of 8s for async API calls. Loading state displayed to user while awaiting response. Prompt length controlled via input truncation and history summarization.
Cost Efficiency	Average token cost per consultation session must remain within budget threshold.	Fixed-length system prompts. Patient history summarized before injection. Adaptive rules are short structured entries, not free-text. Repeated graph KB queries cached for the session duration.
Data Privacy	Must comply with Malaysia's Personal Data Protection Act 2010 (PDPA) for patient health information.	Encryption at rest and in transit. Role-based access control (RBAC). Consent mechanisms for patient data. Audit trail for all AI-generated and doctor-edited records.

3.3. Out of Scope / Future Enhancements

1. E-prescribing with direct pharmacy integration (requires Malaysian Pharmacy Board compliance)
2. Telemedicine / video consultation (significant infrastructure complexity)
3. Insurance claims processing (requires Malaysian insurance provider integration)
4. Medical imaging analysis (specialized AI domain outside workflow automation scope)
5. Full EHR system replacement (CliniFlow AI focuses on workflow automation, not replacing EHRs)
6. Mobile native app (web-responsive design prioritized; native apps considered in future phases)
7. Bahasa Malaysia language support (English-only for MVP; BM support in Phase 2)
8. Multilingual voice transcription including code-switching between English and Malay (English-only transcription for MVP)

---

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

Rollout follows a four-stage canary pattern with quantitative gates between stages:

Stage	Traffic %	Duration	Advancement Gate
Canary	5%	3 days	Error rate <2%, p95 latency <8s, no P1 incident
Beta	25%	1 week	A/B edit-rate improvement over baseline, no P1 incident
Majority	50%	1 week	Doctor NPS >30, no completeness regression
General	100%	—	—

A/B Testing: runs in parallel during the Beta stage. Treatment group gets adaptive-rules-enabled report generation; control group gets baseline prompts without learned rules. Primary success metric: doctor edit rate on generated reports (percentage of characters changed). Target: ≥20% relative reduction for the treatment group after 2 weeks of rule accumulation. Secondary metric: weekly 1-question doctor satisfaction pulse.

4.1.2. Emergency Rollback & Golden Release

The **Golden Release** is the most recent production image tagged as stable in the container registry (tag format: `cliniflow-api:golden-YYYYMMDD`). A new Golden tag is promoted only after a release has passed the Majority stage (50% traffic, no P1 incidents for 7 days).

Rollback Triggers:

- *Automated:* P1 trigger fires (see Priority Matrix) → monitoring stack posts to a rollback webhook → CI/CD pipeline redeploys the Golden image → typical recovery <5 minutes
- *Manual:* On-call engineer invokes rollback via GitHub Actions `workflow_dispatch`. Used for subjective issues not caught by metrics (e.g., clinically inappropriate AI suggestions flagged by doctors)

Graceful Degradation Paths are preserved regardless of rollback state:

- GLM unavailable → Visit module routes to manual-entry template; CRUD continues to work
- STT unavailable → doctor falls back to text input mode
- Neo4j unavailable → agents operate with PostgreSQL-only context (reduced reasoning quality, system remains functional)

4.1.3. Priority Matrix

Priority	Trigger Condition	Response SLA	Action	Escalation
P0 Catastrophic	Patient data leak, unauthorized access, or data corruption detected	Immediate	Full system freeze, forensic snapshot, notify stakeholders and legal	Phone + email + incident war room
P1 Critical	GLM error rate >5% for 5 min, or clinically dangerous content generated	<15 min	Automated rollback to Golden image. Disable AI agents.	PagerDuty / phone
P2 High	Medical report accuracy drops below 70% (edit rate), or p95 latency >10s	<1 hour	Switch to cached/simplified prompts. Alert engineering.	Slack @here
P3 Medium	Adaptive rule proposals rejected by	<24 hours	Pause adaptive rule learning. Review rule generation logic.	Slack channel, daily digest

	doctor >50% of the time			
P4 Low	Minor UI inconsistencies, non-critical feature requests	Next sprint	Log to backlog.	GitHub issue

4.2. Monitoring

4.2.1. Monitoring Stack

CliniFlow AI uses a hosted SaaS-first stack to minimize ops overhead during pilot deployment:

Tool	Purpose	Cost
Grafana Cloud (free tier)	Metrics via Prometheus remote write, logs via Loki, traces via Tempo, unified dashboards	$0 up to 10k series, 50GB logs
Sentry (free tier)	Error tracking with stack traces, release tagging, user session capture	$0 up to 5k events/month
Uptime Robot	External HTTP health checks every 5 min from multi-region probes	$0 up to 50 monitors

Self-hosted fallback: the same stack (Prometheus + Grafana + Loki) can be run via Docker Compose on the same host. This path is chosen only if PDPA review requires all telemetry to remain on-premise.

4.2.2. Four Golden Signals per Service

Each service instruments the same four signals (after Google SRE) so alerting and dashboards are consistent across the Java/Python boundary:

Signal	Spring Boot	Python Agent
Latency	p50/p95/p99 per endpoint via Micrometer + Actuator	p50/p95/p99 per agent via prometheus_client
Traffic	Requests/sec per endpoint	Agent invocations/sec, GLM calls/sec, STT calls/sec

Errors	HTTP 5xx rate, uncaught exceptions	Failed agent runs, GLM errors, STT errors, parse failures
Saturation	JVM heap, DB connection pool, thread pool queue depth	asyncio event loop lag, Neo4j connection pool

4.2.3. Planned Dashboards

Four Grafana dashboards mapped to operational concerns:

1. *System Health* — uptime, error rates, and latency for both services at a glance. First dashboard on-call opens during an incident.
2. *AI Pipeline* — GLM request latency, token usage, cost per consultation, error rate per agent type, STT transcription confidence distribution.
3. *Clinical Quality* — adaptive rule proposal acceptance rate, medical report edit rate trend, missing-info detection hit rate, drug interaction warnings surfaced.
4. *Data Layer* — PostgreSQL query latency by table, Neo4j query latency (including the drug interaction multi-hop query specifically), graph size growth over time.

4.2.4. Alert Routing

Alerts are defined in Grafana Cloud using Prometheus-style rules and routed by priority:

- *P0/P1* → PagerDuty / direct phone call (treated like an on-call shift during pilot)
- *P2* → Slack or Discord channel with @here mention
- *P3* → Channel message without mention + daily digest email
- *P4* → Auto-opened GitHub issue

Concrete alert examples:

- `glm_error_rate > 5% for 5min` → P1
- `p95_latency_seconds{service="visit-agent"} > 10 for 10min` → P2
- `rule_acceptance_rate < 50% over 24h` → P3

4.3. Assumptions

1. Target clinics have reliable internet connectivity for web-based access.
2. Doctors and staff have basic digital literacy and can interact with a web application.
3. Patients have access to a smartphone or computer to interact with the symptom intake chatbot.
4. AI-generated clinical notes always require doctor review and confirmation before finalization — the AI assists but does not replace clinical judgment.
5. English is sufficient for MVP; Bahasa Malaysia support will be added in Phase 2.

6. Z.AI GLM API provides an OpenAI-compatible endpoint that can be called from Python.
7. Voice transcription accuracy of ≥90% is achievable for Malaysian-accented English using an OpenAI Whisper-class Speech-to-Text API. The doctor can always correct the transcript before the agent generates the report.

4.4. External Dependencies

Tools	Purpose	Risks
Z.AI GLM API	Core LLM reasoning engine — handles prompt inference, multi-step reasoning, and generating structured clinical output for all three agents	High — if GLM API rate-limited or unavailable, core AI features fail. Mitigation: graceful degradation to manual entry mode; abstract GLM behind internal API layer for potential provider switch.
Neo4j Community Edition	Graph database for the graph knowledge base — stores patient knowledge graph and adaptive rules	Medium — if Neo4j unavailable, agents lose patient context. Mitigation: cache recent patient context in-memory; system still functions with reduced context quality.
PostgreSQL	Relational database for core application data (users, patients, visits, records)	Medium — critical for all CRUD operations. Mitigation: standard database backup and recovery procedures.
Speech-to-Text API (Whisper-class)	Transcribes live voice recordings and uploaded audio files from consultations before the Visit Agent processes them.	Medium — if STT unavailable or low-quality, doctors fall back to manual text input for that consultation. Mitigation: abstract STT behind an internal interface; text-input path always available as fallback.

4.5. DevOps & Deployment

This section captures the operational practices that make CliniFlow AI deployable, observable, and recoverable. Monitoring is covered in Section 4.2; this section covers everything else.

Schema management — important correction: Flyway was initially planned but removed because it is incompatible with Supabase's pgbouncer transaction-mode pooler (prepared-statement cache collisions on boot). The

`backend/src/main/resources/db/migration/*.sql` files remain in the repo as **reference documentation only** — they are never executed automatically. Schema changes are applied **manually via the Supabase SQL editor** (or `supabase db push`) in the same version order (V1 → V8). JPA is configured with `ddl-auto: none` so Hibernate never touches DDL. Neo4j schema (constraints + indexes) is applied at FastAPI startup from `agent/app/graph/schema.py::apply_schema`, using idempotent `IF NOT EXISTS` statements.

4.5.1. CI/CD Pipeline

Platform: GitHub Actions (free tier covers hackathon and pilot-scale usage).

Pipeline stages:

1. *Lint & format* — Checkstyle/Spotless for Java, Ruff/Black for Python. Fast-fail.
2. *Test* — JUnit 5 + Testcontainers for Spring Boot integration tests, pytest for Python unit tests.
3. *Build & scan* — Maven package, Docker image build, Trivy scan for known CVEs in base images.
4. *Publish* — Tag and push images to GitHub Container Registry with both `latest` and `sha-{commit}` tags.
5. *Deploy* — On push to `main`, auto-deploy to staging. Manual `workflow_dispatch` promotes staging to production.

Branch protection on `main`: require PR review, require CI green, disallow force pushes. Not strictly necessary for a solo developer, but signals production-readiness and catches mistakes.

4.5.2. Containerization Strategy

Both services ship as multi-stage Docker images:

- *Spring Boot:* `eclipse-temurin:21-jdk` for build, `eclipse-temurin:21-jre-alpine` for runtime. Target image size ~200MB.
- *Python agent:* `python:3.12-slim` base with multi-stage wheel caching. Target image size ~300MB.

Three docker-compose files:

- `docker-compose.yml` — base definition of services, networks, volumes
- `docker-compose.dev.yml` — local overrides (hot reload, exposed ports, source volume mounts)
- `docker-compose.prod.yml` — production overrides (resource limits, restart policies, prod env vars)

Nginx reverse proxy fronts both services with TLS termination and path-prefix routing.

4.5.3. Logging & Correlation

Both services emit structured JSON logs to stdout, captured by Docker and shipped to Grafana Loki. Each log line carries:

- `timestamp` (ISO 8601)
- `level` (INFO/WARN/ERROR)
- `service` (spring-boot | python-agent)
- `correlation_id` — generated at the API gateway on every incoming request, propagated via the `X-Correlation-ID` HTTP header to the Python agent and included in every downstream log line
- `visit_id`, `doctor_id`, `patient_id` where applicable (hashed in non-audit logs)

This makes end-to-end tracing across the polyglot boundary a one-line query: `{correlation_id="abc123"}` in Loki returns every log emitted for that consultation by either service.

Separate PDPA audit log: append-only, written to its own PostgreSQL table. Records every read and write of patient data with timestamp, actor, action, resource ID. Retained per PDPA retention requirements and never mixed with application logs.

4.5.4. Secrets Management

- *Local development:* `.env.example` committed with placeholder keys; developers copy to `.env` (git-ignored) with local values.
- *CI:* secrets stored as GitHub Actions Secrets, injected as env vars during build/deploy.
- *Production:* docker-compose reads from host environment; production host uses `.env.prod` (file permissions 600, owned by deploy user) as the staging ground.

Never committed: Z.AI GLM API key, STT API key, PostgreSQL credentials, Neo4j credentials, Sentry DSN. Pre-commit hook (detect-secrets) blocks accidental commits of key-like strings.

Future: migrate to a secrets manager (AWS Secrets Manager or HashiCorp Vault) when scaling beyond pilot.

4.5.5. Database Migrations

Flyway is NOT used — it was evaluated and removed. Supabase's pgbouncer pooler (transaction mode) collides with Flyway's use of session-scoped prepared statements during its migration lock, causing the backend to fail at boot. See post-mortem `docs/post-mortem/2026-04-22-backend-boot-and-schema.md` for full detail.

Instead:

- PostgreSQL migrations live in `backend/src/main/resources/db/migration/` as **versioned SQL files for human reference** (`V1__init.sql` through `V8__medical_reports_review_columns.sql`). They are not auto-applied. A developer applies them in order via the **Supabase SQL editor** (or `supabase db push`) whenever the schema changes.
- Spring Boot runs with `spring.jpa.hibernate.ddl-auto: none`. Hibernate must never touch DDL — `validate` was tried and blocked startup whenever a manually-applied migration diverged by a single column.
- Supabase HikariCP is configured with `prepareThreshold: 0` and `preparedStatementCacheQueries: 0` to coexist with pgbouncer transaction mode.
- Two DB-level invariants worth highlighting: `audit_log` and `agent_turns` have BEFORE UPDATE / BEFORE DELETE triggers that raise an exception — they are append-only by design (PDPA). Never UPDATE or DELETE these tables from application code; always INSERT.

For **Neo4j**, the schema (constraints + indexes) is enforced in code at FastAPI startup via `agent/app/graph/schema.py::apply_schema`, invoked from the FastAPI lifespan. All statements use `IF NOT EXISTS` so re-runs are idempotent.

4.5.6. Resilience Patterns

External calls (GLM, STT) use **Resilience4j** in Spring Boot with three patterns:

- *Circuit breaker* — opens if error rate exceeds 50% in a rolling window; half-open after 30s to probe recovery. Prevents cascading failures.
- *Rate limiter* — caps per-doctor request rate to avoid runaway token costs from stuck agents or malformed prompts.
- *Timeout* — strict per-call limits (8s for GLM, 15s for STT). Exceeded timeouts surface as degraded-path fallbacks, not error screens.

The Python agent service uses equivalent patterns via `tenacity` for retries and a lightweight in-memory rate limiter.

---

5. Project Management & Team Contributions

The duration of the preliminary round is approximately 7 days of solo development. Scope is aggressively cut to what can be demoed live end-to-end.

5.1. Project Timeline

Day	Phase	Tasks
1	Planning & Setup	Architecture doc, graph KB schema design, project scaffolding (Spring Boot + FastAPI), GLM API integration test
2	Pre-Visit Agent	Symptom intake chatbot conversation loop, structured pre-visit report generation, write patient data to graph KB
3	Visit Agent (Core)	Voice recording UI + Speech-to-Text integration, medical report generation from transcript, doctor edit UI, graph KB multi-hop query (drug interaction check), save to PostgreSQL
4	Post-Visit + Rule Engine	Post-visit patient summary generation, patient portal view, adaptive rule-proposal logic (scripted demo)
5	Integration & Edge Cases	End-to-end flow integration, edge case handling (ambiguity, missing data, GLM API failure fallbacks)
6	Demo Preparation	Demo script, architecture slides, record backup demo video
7	Buffer & Polish	Bug fixes, UI polish, rehearse presentation

5.2. Team Members Role

Person	Role	Responsibilities
Tang Shao Xian	Solo Developer (Full Stack + AI)	Spring Boot backend (DDD bounded contexts), FastAPI agent service (LangGraph), React frontend, graph KB schema design, adaptive rule engine, GLM integration, system documentation

5.3. Recommendations

1. **Redis Cache** for frequently accessed patient context from the graph KB, reducing Neo4j query load during peak hours.
2. **Prompt versioning** to track and rollback prompt templates when GLM output quality degrades.

3. **Structured logging with correlation IDs** across Spring Boot and Python services to trace a single consultation request end-to-end.
4. **Adaptive rule confidence scoring** — only apply rules that have been validated by doctor acceptance above a threshold (e.g., >80% acceptance rate).